
CMPS432 Fall 2025 Programming Assignment 2.

Assigned: Friday, October 31, 2025

Due: **Monday, December 1, 2025** (by midnight, submit a package of codes in .ipynb and a report in .pdf via Canvas)

Maximum: 120 points (out of 100 points, 20 points from bonus tasks)

Note: This assignment is to be done by an individual student, teamwork is NOT allowed. Use of services/helps from the Internet/Web/GenAI on the assignment is NOT allowed.

In this assignment, you will implement the k-Means algorithm and use your implementation, along with the k-Means available from sklearn, to cluster the *Credit Card Dataset* and perform cluster analysis.

For implementation, a template for k-Means is given for your reference. After running k-Means on the dataset to generate clusters, you need to perform a cluster analysis on the original dataset to discuss the results and report your findings.

Dataset and features

- *Credit Card Dataset.* Experimentally, you will use all numerical features, i.e., Age, LIMIT_BAL, BILL_AMT 1..6, and PAY_AMT 1..6 (14 features in total).

Implementation and Experiments

Task 1. Data Preprocessing [12 points]

- a) As 14 attributes are used for clustering (which is quite many), please apply *Principle Component Analysis* (PCA) from sklearn to reduce the dimensionality to 3 (i.e., n_components=3). The dataset to run k-Means therefore has 24000 rows and 3 columns. Note that normalizing the attributes by *standardization* before PCA is a common practice. Please perform the standardization in this task. Please show the result of the first 5 records in your report. [7 points]
- b) Before you run k-Means, please perform *normalization* on the attributes used for clustering. Describe the normalization scheme you adopt and show the result of the first 5 records in your report. [5 points]

Task 2. (Your own) implementation of the k-Means Algorithm [\[40 points\]](#)

k-Means is a clustering algorithm widely used for explorative analysis of unknown (unlabeled) data. The goal is to find k clusters in the data, where the number of clusters k is a parameter to be specified. The algorithm works iteratively to assign each data point to one of the k clusters based on the considered features/attributes of the data points. The results of the k-Means clustering algorithm are (1) the *centroids* of the k clusters, which can be used to denote (label) their corresponding clusters; (2) the clustering label (or cluster number) for each data point which is assigned to the corresponding cluster.

a) k-Means [\[22 points\]](#)

Please follow the instructions below, step by step, to implement your own k-Means in python. A function template is provided below, with codes for key functions, i.e., `rand_center(data, k)`, `update_centroids(data, centroids, k=5)` and `converged(centroids1, centroids2)`. Please implement these functions in Jupyter Notebook:

```
def rand_center(data,k):
    """
    >>> Function you need to write
    >>> Select "k" random points from "data" as the initial centroids.
    """
    pass

def converged(centroids1, centroids2):
    """
    >>> Function you need to write
    >>> check whether centroids1==centroids2
    >>> add proper code to handle infinite loop if it never converges
    """
    pass

def update_centroids(data, centroids, k):
    """
    >>> Function you need to write
    >>> Assign each data point to its nearest centroid based on the Euclidean distance
    >>> Update the cluster centroid to the mean of all the points assigned to that cluster
    """
    pass
```

```

def kmeans(data,k=5):
    """
    >>> Main function of your k-Means implementation
    """
    # step 1:
    centroids = rand_center(data,k)
    converge = False
    while not converge:
        old_centroids = np.copy(centroids)
        # step 2 & 3; labels can be an array of labels for all the data points
        centroids, label = update_centroids(data, old_centroids)
        # step 4
        converge = converged(old_centroids, centroids)
    print(">>> final centroids")
    print(centroids)
    return centroids, label

```

Step 1. Let $C = \{c_1, c_2, c_3, \dots, c_k\}$ denote the set of centroids. Randomly pick k data points from the 24000 data points as the initial cluster centroids.

Step 2. Assign each data point x to its nearest cluster, based on its Euclidean distances to the centroids. Effectively, this is to assign the cluster number/label i to x in accordance with the following:

$$\arg \min_{c_i \in C} \text{dist}(x, c_i)$$

where $\text{dist}(\cdot)$ denotes the Euclidean distance.

Step 3. Update each cluster centroid to the mean of all the points assigned to the corresponding cluster.

$$c_i = \frac{1}{|S_i|} \sum_{x \in S_i} x$$

where S_i is the set of points assigned to i -th cluster.

Step 4. Repeat Step 2 and Step 3 until the centroids converges. (Note: add proper code to handle infinite loop if it never converges.)

b) Bisecting k-Means [8 points]

As discussed in classes (also refer to the textbook), the bisecting k-Means algorithm can be implemented using the basic k-Means algorithm. As part of the assignment, you need to implement bisecting k-means using your implementation. As there are many ways to choose a cluster to split, e.g., the largest cluster in each step, *please specify your approach clearly in the report*. In addition to the clusterings generated by the basic k-Means, you also need to run the bisecting k-Means to generate clusterings for comparison.

c) Evaluation Metrics [10 points]

As discussed in classes, *SSE* is used to decide the best clustering. As part of the assignment, you need to implement *SSE(centroids, label, data)*. Feel free to change the parameters list of *SSE* as you see appropriate, e.g., using *cluster number* instead of *label*. Note that K-Means should be run on the dataset multiple times to obtain a set of clusterings, from which you will apply *SSE* to decide the best clustering for cluster analysis.

You are required to use *SSE* as the default metric. On other hand, if you find other useful metrics good for decide the best clustering (in different aspects), please feel free to use them for comparison with the clustering obtained via *SSE*. Note that extra effort may potentially earn you extra credit, but please point it out explicitly in your report!

In Task 2(a)-(c), you need to submit your runnable implementation code and result in Jupyter Notebook. The code should be well formatted and well documented, without leaving any unnecessary test code or commented-out print statements in the final submission. This requirement is factored in the grading.

Task 3. Cluster Analysis [48 points]

After the clustering is decided, an important task is to perform data analysis on data objects in each cluster. The goal is to characterize and understand better the clients in different clusters. In addition to *DEFAULT* (i.e., the class label used in the classification task), the analysis may be carried out on all other attributes/features to characterize the clusters. You may examine the attributes of the medoid (i.e., the representative client) in each cluster or calculate the statistics of user attributes in each cluster for comparison. For example, you may calculate the Impurity (GINI Index) of the clients' education (and marital, etc.) or calculate the average limited balance amount (and age, etc.) for each cluster. You may also analyze the distributions of various attribute values to see whether

you can capture a pattern. There is various interesting information about clients in different clusters that you can explore (i.e., the more the better) and make comparison. Please show your findings in the follow-up tasks/discussions.

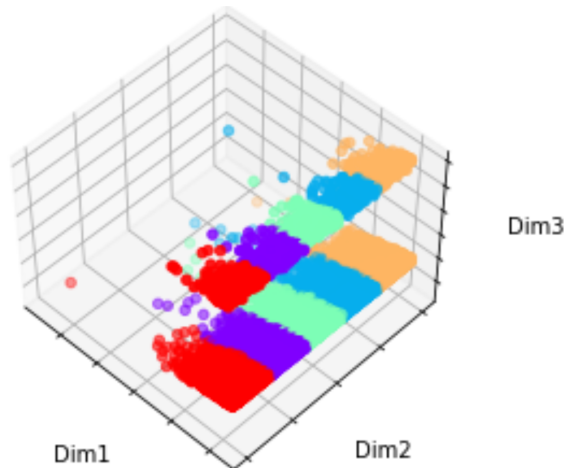
Experimental Practice

The assignment consists of various steps that need to run your code multiple times. *A common and very useful practice in data mining (and in computer science in general) is to use scripts to make your code execution easy.* If you exercise this practice, some repetitive tasks will be very simple.

Results and Follow-up Discussion

- a) Given $k = 5$, run your own k-Means multiple times (at least 20 times) and bisecting k-Means (at least 5 trials in each split) to find the best corresponding clusterings based on SSE (sum of squared errors). Compare them with the result obtained from k-Means in sklearn (with `n_clusters=5`). Show all these *three* clustering results using matplotlib. You should plot 3D graph (e.g., Principal component 1 vs Principal component 2 vs Principal component 3.) For the plot, you can refer to the sample code from sklearn [here](#). An example figure with $k=5$ is shown below (Note that the figure is for illustration only. It's generated with a different dataset, so your plots should be very different!). [15 points]

k-Means clustering with $k=5$ (each color belongs to a cluster)



- b) Among the three best clusterings you obtained via scikit-learn, your own k-Means, and bisecting k-Means implementations, which clustering is the best? Use the results obtained in (a) to explain why it is important to choose proper initial centroids. Please note that your k-Means and bisecting k-Means implementations use *random initialization*, while sklearn uses a default initialization method from k-Means++. [6 points]

- c) Please report your findings in cluster analysis. For example, based on the best clustering you obtained, *compare clusters in terms of (i) Impurity (Gini Index) of DEFAULT and (ii) percentage of 'yes' in DEFAULT*. Which measure can reveal more interesting information under the context of this application? You should make comparison in terms of the characteristics and distribution of data points in those clusters. In the report, show your result and findings with explanation. Illustrations with tables and figures are excellent.

In addition to the class label, i.e., DEFAULT, other attributes, may also be interesting to analyze on. Please exercise the concepts and skills you learned in the course in the comparison. Note that simply completing the comparison mentioned in the above example, i.e., DEFAULT, is insufficient. This task will be graded based on both of the effort you made on the analysis and the findings. Please make as many analyses as you can and report your findings. [15 points]

- d) You are asked to perform k-Means with $k=5$ in (a). However, k is a parameter that can be further explored. By trying different k , you may get totally different clustering results. Please run k-Means with k in different ranges (e.g., from 2, 3, ... to 10 or from 5, 10, 15, ..., to 50). Please keep track of SSE for each k (e.g., a line graph) and decide the best k . You should present the best k and state your reasoning. Finally, please report your findings in cluster analysis for this newly obtained clustering in comparison with the result obtained in (c). [12 points]

Bonus Tasks [20 points].

1. **Bonus (6%):** In the preprocessing step, the data points are normalized. Is it really better than clustering without preprocessing with normalization? Please use data without normalization to find the best clustering. Compared with the normalized data, present the plots to make comparison. Report your finding in the comparison. (Note: you are not required to perform clustering analysis in this task but you are welcome to do so.)
2. **Bonus (14%):** You may repeat the above (or part of the above) tasks by using your choice of attributes for clustering. State attributes you propose to use (with justification/explanation) and report your findings/results in comparison with the previously reported result. For example, using only Age, LIMIT_BAL and PAY_AMT (in month of your choice) as a combination of features can result in a different clustering. Please perform clustering analysis on the new clustering and compare with what you have previously. Please report your findings and explain different clustering results.

Submission Instructions

You have two options for submission on Canvas:

Option 1 – Two Files:

- Submit (1) a self-contained PDF report, and (2) the complete Jupyter Notebook (.ipynb).
- The PDF report must clearly present all experiment results and corresponding discussions for every task.
- The Jupyter Notebook may include code, outputs, and brief comments, but the main analysis and explanations must appear in the report.

Option 2 – Single Notebook:

- Submit only one .ipynb file which is well-organized and clearly written.
- Each task must be clearly labeled, and both the results and discussions for each task should be presented within the notebook.
- In this case, a separate PDF report is not required.

Important Note:

- Regardless of which option you choose, your submission must be self-contained.
 - *Do not expect the TA to cross-check the submitted files for you.* Thus, all experiment results and corresponding discussions for every task should be reported in ONE file, i.e., either in a pdf file or Jupyter Notebook. Do not split experiment results and discussions between files.
 - Submissions that are incomplete or fragmented will incur a grade penalty.
-

Packages

- sklearn (<http://scikit-learn.org/>). A machine learning framework in Python
- matplotlib (<https://matplotlib.org/>). Website provides tutorials on how to plot bar chart and histogram in Python.
- NumPy (<https://numpy.org>). A fundamental package for scientific computing in Python.
- pandas (<https://pandas.pydata.org/>). A framework for easy-to-use data analysis and manipulation in Python